지능형 DB 13주

PL/SQL (PROCEDURE, FUNCTION, TRIGGER)

PL/SQL 개요

Procedure Language

비절차적 언어인 SQL에 프로그래밍적 요소(예, 변수사용 등)를 가미해서 좀 더 다이나믹하게 프로그래밍 할 수 있도록 하는 기능

엑셀의 매크로와 비슷한 기능

일단 한번 만 구현하면 여러 번 반복 사용가능

한 건만 처리하는 'procedure'와 여러 건을 처리하는 'cursor'로 구분한다.

프로시저와 일반 PL/SQL 블럭

NON-프로시저	프로시저
매번 컴파일되어 실행	한번만 컴파일하여 실행
DB 저장안됨	DB에 저장됨 (프로시저 이름 저장)
다른 앱에서 호출 안됨	다른 앱에서 호출 가능
매개변수 사용 안됨	매개변수 사용 가능
결과값 반환 안됨	함수의 경우 결과값 반환

프로시저 변수형

스칼라 변수

예) p_ename varchar2(10);

참조형 변수

예) p_ename emp.ename%type;

테이블 전체 구조 변수

예) p_emp dept%rowtype;

%ROWTYPE

```
DECLARE
v_dept        dept%rowtype; /* dept테이블 구조 전체를 그대로 적용 */
BEGIN
select * into v_dept
from dept
where dept = 10;
dbms_outout.put_line ( v_dept.deptno v_dept.dname);
```

(STORED) PROCEDURE 작성

CREATE or REPLACE PROCEDURE 프로시저명

CREATE PROCEDURE 프로시저명 /*프로시저명은 unique해야 한다. 중복안됨*/

중복이 될 경우 덮어 쓰고 싶다면?

create or replace procedure ~

예제1

부서번호가 '30'인 사원의 job을 'MANAGER'로 업데이트하는 프로시저를 작성하라.

```
create or replace procedure pre_job
is begin
update emp set job = 'MANAGER' where deptno = 30;
end;
/
SQL> exec pre_job; /* 확인은 select job from emp where deptno = 30; */
```

예제2

사번을 입력받아 급여를 2000으로 인상하는 프로시저를 작성해보자. (사번 : 7900)

```
create or replace procedure pre_sal
( v_empno emp.empno%type) /* 입력 사번 저장 변수 */
is begin
 update emp set sal = 2000
 where empno = v_empno;
 end;
/        /* 실행은 exec pre_sal(7900); */
```

OUTPUT 변수

프로시저에 실행 결과값을 변수에 저장하여 다른 프로시저에서 사용

```
create or replace procedure temp_emp
( v_empno     in      emp.empno%type , /* 'in'은 생략가능 */
  v_ename     out     emp.ename%type ,
  v_sal       out     emp.sal%type)
is
begin /* 다음장에 계속 */
```

```
 select ename, sal into v_ename, v_sal
 from emp
 where empno = v_empno;

end;
/
/* v_ename   (in)      emp.ename%type , in이 없으면 디폴트 'in' */
```

```
set serveroutput on
declare
v_ename        emp.ename%type;
v_sal          emp.sal%type;
begin
temp_emp(7900, v_ename, v_sal);
dbms_output.put_line ( v_ename || '    ' || v_sal);
end;
/
```

다른 방식

```
set serveroutput on
declare
v_ename      emp.ename%type;
v_sal        emp.sal%type;
begin
temp_emp(v_empno=>7900, v_ename=>v_ename, v_sal=>v_sal);
dbms_output.put_line ( v_ename || '   ' || v_sal);
end;
/
```

FUNCTION (사용자 정의 함수)

```
 create or replace FUNCTION 함수명
변수명          변수타입(크기)
RETURN 데이터 타입
IS
BEGIN
   SQL문장
   RETURN 변수명;
END;
```

예제

부서번호를 입력받아 해당 부서의 최고 급여액을 알아보는 사용자 정의 함수를 작성해 보자.

```
create or replace fuction fn_maxsal
  ( v_deptno  emp.deptno%type )
return          number /* 리턴 데이터의 타입만 정의한다. */
is
        max_sal emp.sal%type;
BEGIN /* 다음 장에 계속 */
```

```
 select max(sal) into max_sal
 from emp
 where deptno = v_deptno;
 RETURN max_sal; /* 이전 장의 return 타입과 동일해야 한다. */
END;
/
SQL> select max_sal(10) from emp;
```

작성된 max_sal() 함수를 이용하여 부서별 최고 급여액을 구하는 SQL을 작성하시오.

```
select deptno, max_sal(deptno)
        from emp
        group by deptno;
```

TRIGGER

트리거(trigger)란 데이터의 변경(INSERT, DELETE, UPDATE)문이 실행될 때 자동으로 실행되는 프로시저를 말한다. 트리거를 설정해 놓으면 데이터가 변경되기 전(BEFORE)과 후(AFTER) 시점에 개입하여 원하는 로직을 실행할 수 있다. 트리거가 프로시저와 다른 점은 프로시저는 호출을 해야지 실행이 된다면 트리거는 조건이 충족되면 자동으로 실행된다는 것이다.

출처: https://generalcoder.tistory.com/6 [HighGarden:티스토리]

기본 형태

```
CREATE [OR REPLACE] Trigger 트리거명
Before (or After) UPDATE OR DELETE OR INSERT ON 테이블명
[FOR EACH ROW]
DECLARE
--변수선언부
BEGIN
  --프로그램 코딩부
END;
```

예제

```
SQL> grant create trigger to SCOTT;

CREATE OR REPLACE TRIGGER 트리거명
before/after insert or update or delete on 테이블명
for each row /* 각 행마다 실행 */
declare
 필요 변수 선언
```

```
BEGIN
IF inserting /* 새로운 값 입력 */
  and :new.deptno is NULL
THEN
  :new.deptno := 1000;
END IF;
END;
/
```

```
insert into dept ( dname, loc )
values ( 'ENG', 'SAN JOSE');

select * from dept;
```

예제 2

신규 테이블 생성해보자.

```
create table temp_order
(  ordno        number,
   ordname      varchar2(10),
   orddate      date);
```

특정 시간에만 입력이 가능하도록 트리거 생성

```
create or replace trigger time_check
before insert on temp_order
begin
if (to_char(sysdate, 'HH24:MI) not between '실습시간1' and '실습시간2')
then
```

```
RAISE_APPLICATION_ERROR ( -20200, '입력시간 오류');
END IF;
END;
/
```

RAISE_APPLICATION_ERROR : 사용자 정의 오류

-20200 : -20000 ~ -20999 범위에 한해 정의할 수 있다.

실습시간에 맞추어 입력을 실행하고 시간 범위를 벗어난 후 다시 입력해 보자.

insert into temp_order values (1, ‘ORANGE’ SYSDATE);

트리거에서 정한 시간 후 입력

insert into temp_order values (2, ‘APPLE’ SYSDATE);

특정 제품만 입력이 가능하도록 하고 싶다면..

```
create or replace trigger tri_product
before insert on temp_order
for each row
when ( new.ordname = 'PINE') /* 'pine'만 특정 시간에 입력 */
BEGIN ~
```